Python Lesson

Mastering Python Dictionaries

A Complete Beginner-to-Intermediate Guide

Table of Contents

1. Introduction to Dictionaries

A dictionary in Python is an unordered, mutable collection of key-value pairs. Dictionaries are one of the most powerful and commonly used data structures in Python. They allow you to store and retrieve data efficiently using unique keys rather than numeric indices, making your code more readable and expressive.

Dictionaries are defined using curly braces **{ }**, with each key-value pair separated by a colon. Keys must be immutable types (strings, numbers, tuples), while values can be any Python object. Since Python 3.7, dictionaries maintain insertion order.

> ***Tip:*** *Think of a dictionary like a real-world dictionary: you look up a word (the key) and find its definition (the value). This analogy helps understand why keys must be unique!*

2. Creating and Accessing Dictionaries

Creating a Dictionary

```
# Method 1: Using curly braces
student = {"name": "Alice", "age": 22, "grade": "A"}

# Method 2: Using the dict() constructor
student = dict(name="Alice", age=22, grade="A")

# Method 3: From a list of tuples
student = dict([("name", "Alice"), ("age", 22)])

# Empty dictionary
empty = {}
```

Accessing Values

You can access dictionary values using square bracket notation or the **get()** method. The bracket method raises a **KeyError** if the key does not exist, while **get()** returns **None** (or a custom default) if the key is missing.

```
student = {"name": "Alice", "age": 22, "grade": "A"}

# Bracket notation
print(student["name"]) # Output: Alice
```

```
# Using get() with a default
print(student.get("gpa", 0.0)) # Output: 0.0

# Check if a key exists
if "age" in student:
print(f"Age: {student['age']}")
```

Best Practice: *Use* ***get()*** *when you are unsure whether a key exists. This avoids unexpected crashes from KeyError exceptions in your programs.*

3. Modifying Dictionaries

Dictionaries are mutable, meaning you can add, update, and delete key-value pairs after creation. Here are the most common modification operations:

```
student = {"name": "Alice", "age": 22}

# Add a new key-value pair
student["grade"] = "A"

# Update an existing value
student["age"] = 23

# Delete a key-value pair
del student["grade"]

# Remove and return a value
age = student.pop("age") # age = 23

# Remove last inserted item (Python 3.7+)
last = student.popitem()

# Update multiple values at once
student.update({"age": 24, "gpa": 3.9})
```

Warning: *Using* ***del*** *on a key that does not exist raises a* ***KeyError****. Use* ***pop()*** *with a default value to safely remove keys:* ***student.pop("missing_key", None)****.*

4. Dictionary Methods

Python dictionaries come with a rich set of built-in methods. Here is a reference table of the most frequently used ones:

Method	Description	Example
keys()	Returns all keys	d.keys()
values()	Returns all values	d.values()
items()	Returns key-value tuples	d.items()

get(k, d)	Returns value or default	d.get("x", 0)
setdefault(k, d)	Get or set default value	d.setdefault("x", [])
update()	Merge another dict	d.update({"a": 1})
pop(k)	Remove and return value	d.pop("x")
clear()	Remove all items	d.clear()
copy()	Shallow copy	d.copy()

5. Iterating Over Dictionaries

Looping through dictionaries is a fundamental skill. Python provides several clean ways to iterate over keys, values, or both simultaneously.

```
scores = {"math": 95, "science": 88, "english": 92}

# Iterate over keys (default behavior)
for subject in scores:
print(subject)

# Iterate over values
for score in scores.values():
print(score)

# Iterate over key-value pairs
for subject, score in scores.items():
print(f"{subject}: {score}")

# Sorted iteration
for subject in sorted(scores):
print(f"{subject}: {scores[subject]}")
```

Performance Note: *Calling* ***.items()*** *does not create a copy of the data. It returns a lightweight view object, making iteration memory-efficient even on very large dictionaries.*

6. Dictionary Comprehensions

Dictionary comprehensions provide a concise and expressive way to create new dictionaries from iterables. They follow the pattern: **{key_expr: value_expr for item in iterable if condition}**.

```
# Create a dict of squares
squares = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Filter with a condition
high_scores = {k: v for k, v in scores.items() if v >= 90}

# Transform keys and values
upper_scores = {k.upper(): v for k, v in scores.items()}
```

```
# Swap keys and values
inverted = {v: k for k, v in scores.items()}

# From two lists using zip
names = ["Alice", "Bob", "Carol"]
ages = [25, 30, 28]
people = {name: age for name, age in zip(names, ages)}
```

Tip: *Dictionary comprehensions are not just syntactic sugar. They are often faster than building a dictionary with a traditional for-loop because the comprehension is optimized internally by the Python interpreter.*

7. Nested Dictionaries

Dictionaries can contain other dictionaries as values, creating nested structures. This is extremely useful for modeling real-world data like JSON responses, configuration files, and hierarchical records.

```
# Nested dictionary: a classroom
classroom = {
"student_1": {
"name": "Alice",
"grades": {"math": 95, "science": 88}
},
"student_2": {
"name": "Bob",
"grades": {"math": 78, "science": 92}
}
}

# Access nested values
alice_math = classroom["student_1"]["grades"]["math"]
print(alice_math) # Output: 95

# Safely access with nested get()
grade = classroom.get("student_3", {}).get("grades", {})
print(grade) # Output: {}
```

8. Practical Exercises

Test your knowledge with these exercises. Try to solve each one before looking at any reference material.

Exercise 1 - Word Counter: Write a function that takes a string and returns a dictionary where each key is a word and each value is the number of times that word appears in the string.

Exercise 2 - Merge Dicts: Write a function that takes two dictionaries and merges them. If both contain the same key, add the values together (assume numeric values).

Exercise 3 - Invert Dict: Write a function that swaps keys and values. If multiple keys share the same value, group the original keys into a list.

Exercise 4 - Nested Lookup: Write a function **deep_get(d, keys)** that takes a dictionary and a list of keys, and safely navigates through nested dictionaries returning None if any key is missing.

Exercise 5 - Student Report: Given a nested dictionary of students and their scores, write a function that returns the student with the highest average score.

Congratulations! *You have completed the Python Dictionaries lesson. Dictionaries are a cornerstone of Python programming. Practice the exercises above, and try incorporating dictionaries into your own projects to solidify your understanding.*